

Observation as Computation: GPU Fragment Shaders as Physical Observation Apparatus Through the Triple Equivalence of Oscillatory, Categorical, and Partitional Dynamics

Kundai Farai Sachikonye
AIME Registry for Artificial Intelligence
Experimental Bioinformatics
Maximus-von-Imhof-Forum 3
85354 Freising, Germany
kundai.sachikonye@bitspark.com

April 2, 2026

Abstract

The standard computational pipeline treats GPU fragment shaders as visualization endpoints: computation produces a result, a shader renders it, a human observes the rendering. We prove this view is fundamentally wrong. A fragment shader executing on partition coordinates is not visualizing a result—it is performing a physical observation. The pixel output is not a picture of the categorical state; it *is* the categorical state. We establish this through the Triple Equivalence Theorem, which proves that oscillatory dynamics, categorical state enumeration, and partition operations yield identical entropy $S = k_B M \ln n$ and identical state counts $\Omega = n^M$, making rendering, observation, and computation three descriptions of one act. From this identity we derive: (1) $O(1)$ GPU memory for databases of arbitrary size, since observations can be re-performed on demand and storage is therefore redundant; (2) GPU physical observables—partition sharpness, noise level, phase coherence, interference visibility, multi-resolution consistency—that serve as objective, deterministic training signals requiring no human annotation; (3) a universal training architecture where a compiled probe generates operations, the GPU performs observations as a black-box oracle, and backpropagation passes through the probe alone; (4) hardware requirements reduced from datacenter GPUs (~ 80 GB, $\$500\text{K}$ – $\$5\text{M}$) to any integrated laptop GPU (~ 25 MB working

memory, $\sim \$1\text{K}$). The framework is instantiated for molecular sciences, protein sciences, and microscopy, and extended in principle to genomics, climate science, finance, and general time series. The only domain-specific components are the S-entropy formulas and partition structure; the observation apparatus, training architecture, and physical observables are universal. The shader observes. The observation computes. The computation measures. These are one act.

Keywords: fragment shader, observation apparatus, triple equivalence, S-entropy coordinates, partition observation, GPU physical observables, $O(1)$ memory, compiled probe, categorical state, oscillatory dynamics, training signal, integrated GPU

Part I Foundations

1 Introduction

1.1 The Standard View

The prevailing computational paradigm arranges GPU rendering at the terminus of a linear pipeline:

1. **Computation.** A CPU or GPU kernel produces a numerical result.
2. **Storage.** The result is written to memory.

3. **Rendering.** A fragment shader maps the stored result to pixel colors.
4. **Visualization.** The rendered image appears on screen.
5. **Human observation.** A person looks at the screen and interprets the image.

In this view, the fragment shader is a presentation layer. It adds no scientific content. It is the last step before the human eye, and the human is the true observer. Every textbook on GPU programming, every shader tutorial, every real-time rendering reference [Akenine-Möller et al., 2018, Owens et al., 2007, Harris, 2005] either states or implies this architecture: the shader draws a picture; the picture is for a person to look at.

1.2 Fragment Shaders

We will prove the following: when a fragment shader computes partition state from S-entropy coordinates, the shader execution *is* the observation. The output texture *is* the categorical state, not a picture of it. No human need look at it. No screen need display it. The act of the GPU writing a pixel value for a given partition cell coordinate is, mathematically and physically, the observation of that cell’s state.

The correct pipeline is not a pipeline at all. It is an identity:

$$\text{Rendering} \equiv \text{Observing} \equiv \text{Computing}.$$

The fragment shader is not a visualization tool. It is a *physical observation apparatus*.

1.3 Central Claim

By the Triple Equivalence Theorem [Sachikonye, 2025b,c], oscillatory dynamics, categorical state enumeration, and partition operations are mathematically identical descriptions of any bounded system. The entropy is the same: $S = k_B M \ln n$. The state count is the same: $\Omega = n^M$. The dynamics are the same: partition refinement, categorical enumeration, and oscillatory frequency resolution are three notations for one process.

A GPU fragment shader that maps S-entropy coordinates to partition cell states implements exactly the categorical observation operator $\hat{O}_{\text{cat}}$. Its output texture is the observed partition—not a rendering of the partition, but the partition itself, encoded as pixel values. The rendering operation and the observation operation are the

same mathematical function applied to the same inputs.

This is not a metaphor. It is not an analogy. It is a mathematical identity, and this paper proves it.

1.4 Contributions

This paper makes four contributions:

1. We prove that fragment shaders implement partition observation (Theorem 4.3), establishing the Observation-Rendering Identity.
2. We derive $O(1)$ GPU memory complexity from observation-on-demand (Theorem 6.1), eliminating the need for data storage.
3. We establish five GPU physical observables as a deterministic, objective training signal (Theorem 8.6), replacing human annotation.
4. We demonstrate universality across scientific domains: the only domain-specific components are the S-entropy formulas and partition structure.

The paper is organized in seven parts. Part I establishes the mathematical foundations. Part II proves that fragment shaders are observation apparatus. Part III derives GPU physical observables as training signals. Part IV presents the universal training architecture. Part V instantiates the framework for specific domains. Part VI analyzes performance. Part VII discusses implications and concludes.

2 The Triple Equivalence

We present the Triple Equivalence in full, following Sachikonye [2025b] with complete proofs adapted for the present context.

2.1 Bounded Phase Space

Axiom 1 (Bounded Phase Space Law). Every physical system occupies a phase space volume V satisfying $0 < V < \infty$. That is, there exist constants $V_{\min}$ and $V_{\max}$ such that $0 < V_{\min} \leq V \leq V_{\max} < \infty$ for any realizable configuration of the system.

This axiom is a restatement of physical reality: no system has infinite energy (upper bound) and no system occupies zero volume in phase space (lower bound, from the uncertainty principle or classical thermal fluctuations). The axiom excludes idealized mathematical objects such as point particles with unbounded momentum.

2.2 Oscillatory Necessity

Theorem 2.1 (Oscillatory Necessity). *Any system satisfying Axiom 1 exhibits oscillatory dynamics.*

Proof. Let Γ be the phase space of the system with bounded volume $V < \infty$. By Poincaré’s recurrence theorem [Poincaré, 1890], for any measurable subset $A \subset \Gamma$ with positive measure, almost every trajectory starting in A returns to A infinitely often. Recurrence in a bounded space implies quasi-periodic dynamics: the trajectory revisits neighborhoods of its initial condition, oscillating through the accessible region of phase space.

Formally, let ϕ_t be the time-evolution map. For almost every initial condition $x_0 \in \Gamma$, there exists a sequence of recurrence times $\{t_k\}_{k=1}^\infty$ with $t_k \rightarrow \infty$ such that $\|\phi_{t_k}(x_0) - x_0\| < \epsilon$ for any $\epsilon > 0$. The trajectory oscillates: it departs from x_0 , explores the bounded region, and returns. The Koopman operator $U_t f = f \circ \phi_t$ acting on $L^2(\Gamma)$ has a discrete spectrum of frequencies $\{\omega_j\}$ characterizing these oscillations [Koopman, 1931]. The system is irreducibly oscillatory. $\square$

2.3 The Triple Equivalence Theorem

Theorem 2.2 (Triple Equivalence). *For any bounded oscillatory system with M modes, each quantized into n levels, the following three descriptions are mathematically equivalent:*

1. **Oscillatory description.** *The system is a collection of M oscillators, each in one of n states. The entropy is $S_{\text{osc}} = k_B M \ln n$.*
2. **Categorical description.** *The system state is an M -tuple from an n -letter alphabet. The state count is $\Omega_{\text{cat}} = n^M$ and the entropy is $S_{\text{cat}} = k_B \ln \Omega_{\text{cat}} = k_B M \ln n$.*
3. **Partition description.** *The phase space is partitioned into n^M cells, each labeled by an M -digit base- n address. The entropy is $S_{\text{part}} = k_B \ln n^M = k_B M \ln n$.*

All three yield the same entropy $S = k_B M \ln n$ and the same state count $\Omega = n^M$.

Proof. We establish bijections between the three descriptions.

Oscillatory $\rightarrow$ Categorical. Each of the M oscillators is in one of n states. Assign to each oscillator state a symbol from an n -letter alphabet $\{0, 1, \dots, n-1\}$. The complete system state is then an M -tuple $(a_1, a_2, \dots, a_M) \in \{0, \dots, n-$

$1\}^M$. This is a bijection: every oscillatory configuration maps to exactly one M -tuple, and vice versa.

Categorical $\rightarrow$ Partition. Each M -tuple $(a_1, \dots, a_M)$ defines a unique address in an M -level partition of phase space. The first digit a_1 selects one of n coarse cells; within that cell, a_2 selects one of n sub-cells; and so on to depth M . The resulting address uniquely identifies one of n^M cells. This is a bijection: each M -tuple maps to exactly one cell, and each cell is addressed by exactly one M -tuple.

Partition $\rightarrow$ Oscillatory. Each cell in the partition corresponds to a specific oscillatory configuration: the j -th digit of the cell address gives the state of the j -th oscillator. This closes the cycle of bijections.

Since the bijections are measure-preserving (each cell has equal measure under the microcanonical assumption), the Boltzmann entropy computed in each description is:

$$S = k_B \ln \Omega = k_B \ln n^M = k_B M \ln n. \quad \square$$

Corollary 2.3 (Rendering-Observation-Computation Identity). *Rendering partition cells (writing pixel values at cell coordinates), observing categorical states (reading the symbol at each cell), and measuring oscillatory behavior (determining the frequency state of each mode) are three descriptions of one operation.*

Proof. By Theorem 2.2, the three descriptions are bijective. An operation that maps cell coordinates to state values implements all three simultaneously. A fragment shader that takes UV coordinates (cell addresses) and outputs pixel values (state symbols) is therefore simultaneously rendering, observing, and measuring. $\square$

Corollary 2.4 (Processor-Oscillator Duality). *Every oscillator is a processor. Every processor is an oscillator.*

Proof. By Theorem 2.1, every bounded physical system oscillates. By Theorem 2.2, each oscillatory state is a categorical state—an element of a finite alphabet. A device that transitions between categorical states according to rules is a processor (a finite state machine). Conversely, every physical processor is a bounded system and therefore oscillates by Theorem 2.1. The processor and the oscillator are dual descriptions of the same physical object. $\square$

2.4 Commutation of Observation Operators

Theorem 2.5 (Categorical Observation Commutation). *Let $\hat{O}_{\text{cat}}$ be the categorical observation operator (mapping cell addresses to categorical states) and $\hat{O}_{\text{phys}}$ be a physical observation operator (measuring a continuous observable). Then $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$: categorical observation is quantum non-demolition with respect to physical observables.*

Proof. The categorical observation operator $\hat{O}_{\text{cat}}$ reads the partition cell address of the system state. This is a coarse-grained projection: it determines which cell the system occupies without disturbing the system’s position within the cell. Formally, $\hat{O}_{\text{cat}}$ is a projection operator onto the subspaces defined by the partition cells:

$$\hat{O}_{\text{cat}} = \sum_{j=1}^{n^M} |c_j\rangle\langle c_j|, \quad \text{where } |c_j\rangle \text{ spans cell } j.$$

A physical observable $\hat{O}_{\text{phys}}$ that is constant within each cell (i.e., $\hat{O}_{\text{phys}}$ is a function of the cell label, not of the position within the cell) commutes with $\hat{O}_{\text{cat}}$:

$$[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = \hat{O}_{\text{cat}}\hat{O}_{\text{phys}} - \hat{O}_{\text{phys}}\hat{O}_{\text{cat}} = 0,$$

because both operators are diagonal in the cell basis. This is precisely the condition for quantum non-demolition measurement [Braginsky and Khalili, 1992]: observing the categorical state does not perturb the physical observable, and measuring the physical observable does not alter the categorical state. The observation can be repeated without degradation. $\square$

Remark 2.6. Theorem 2.5 is the reason GPU observations are repeatable: reading the partition state via a fragment shader does not alter the state. The same observation can be performed again with identical results. This is not a property of deterministic software—it is a consequence of the mathematical structure of categorical observation.

3 S-Entropy Coordinate Space

Definition 3.1 (S-Entropy Coordinates). For any bounded oscillatory system, the *S-entropy coordinate* is the triple $(S_k, S_t, S_e) \in [0, 1]^3$, where:

- S_k (knowledge/kinetic): encodes which partition cells are occupied. This is the spatial structure of the state—the distribution of excitation across modes.
- S_t (temporal): encodes the oscillatory frequency structure. This is the temporal fingerprint—the spectrum of frequencies present in the system’s dynamics.
- S_e (evolution): encodes the partition depth or refinement level. This is the resolution at which the system is observed—how many levels of the partition hierarchy have been resolved.

The S-entropy space $\mathcal{S} = [0, 1]^3$ is universal: every bounded oscillatory system maps to a point in $\mathcal{S}$. The specific formulas for (S_k, S_t, S_e) differ by domain, but the structure is invariant.

Definition 3.2 (Domain-Specific S-Entropy Maps). The S-entropy map $\sigma_D : \text{Domain}_D \rightarrow \mathcal{S}$ for domain D is defined as:

- **Molecular:** S_k from vibrational mode amplitudes, S_t from vibrational frequencies, S_e from number of resolved modes.
- **Protein:** S_k from amino acid physico-chemical properties (hydrophobicity, volume, charge), S_t from backbone oscillation frequencies, S_e from hierarchical structural depth (residue $\rightarrow$ secondary $\rightarrow$ domain $\rightarrow$ global).
- **Imaging:** S_k from pixel intensity distributions, S_t from spatial frequency content, S_e from multi-scale resolution depth.
- **General:** S_k from the amplitude spectrum of the system’s state, S_t from the frequency spectrum, S_e from the number of resolved partition levels.

Proposition 3.3 (Universality of S-Entropy Structure). *For any two bounded oscillatory systems A and B in potentially different domains, the categorical distance*

$$d_{\text{cat}}(A, B) = \sqrt{(S_k^A - S_k^B)^2 + (S_t^A - S_t^B)^2 + (S_e^A - S_e^B)^2}$$

is a metric on $\mathcal{S}$ and provides a domain-independent measure of dissimilarity.

Proof. This is the Euclidean metric on $[0, 1]^3$, which satisfies non-negativity, identity of indiscernibles (since different systems occupy different points in $\mathcal{S}$ at sufficient resolution [Sachikonye, 2025b]), symmetry, and the triangle inequality.

The domain independence follows from the universality of the S-entropy structure: the coordinates (S_k, S_t, S_e) have the same mathematical meaning regardless of the physical system they describe. $\square$

Part II

The Fragment Shader as Observation Apparatus

4 Why Fragment Shaders Are Observation Apparatus

4.1 The Fragment Shader as a Function

Definition 4.1 (Fragment Shader). A *fragment shader* is a function $f : \mathbb{R}^2 \times \mathcal{U} \rightarrow \mathbb{R}^4$ that maps a two-dimensional coordinate (the UV or texture coordinate) together with a set of uniform parameters $\mathcal{U}$ to a four-component output (typically RGBA color). The function is executed in parallel for every pixel in the output texture. The GPU hardware guarantees that:

1. Each invocation is independent (no communication between fragments).
2. All invocations execute the same program (SIMD parallelism).
3. The output is deterministic for fixed inputs.

In the standard rendering pipeline [Akenine-Möller et al., 2018, Khronos Group, 2012], the fragment shader’s output is interpreted as a color value to be displayed on screen. But nothing in the mathematical definition requires this interpretation. The output is a four-component vector. Its meaning is determined by the program that wrote it and the system that reads it.

4.2 The Observation-Rendering Identity

Definition 4.2 (Partition Observation). A *partition observation* is a function $O : \{1, \dots, n^M\} \rightarrow \{0, \dots, n-1\}^M$ that maps each cell in a partition to the categorical state occupying that cell. The domain is the set of cell addresses; the range is the set of categorical states.

Theorem 4.3 (Observation-Rendering Identity). *Let f_{shader} be a fragment shader that computes partition state from S-entropy coordinates:*

$$f_{\text{shader}}(\mathbf{uv}, \mathcal{U}) = \text{partition_state}(S_k(\mathbf{uv}, \mathcal{U}), S_t(\mathbf{uv}, \mathcal{U}), S_e(\mathbf{uv}, \mathcal{U}))$$

Then the render operation $\mathcal{R}$, which evaluates f_{shader} at every pixel, is identical to the observation operation $\mathcal{O}$, which determines the categorical state of every partition cell:

$$\mathcal{R} = \mathcal{O}.$$

The output texture T is the categorical state of the partition, not a picture of it.

Proof. We prove this by exhibiting a bijection between the rendering operation and the observation operation.

Step 1: Domain correspondence. The render operation evaluates f_{shader} at each pixel coordinate $\mathbf{uv} \in \{(u_i, v_j)\}_{i,j}$. Under the partition encoding, each pixel coordinate maps to a cell address: $\mathbf{uv} \mapsto (a_1, a_2, \dots, a_M)$ where the digits are determined by the UV coordinate’s position in the partition grid. This establishes a bijection between pixel coordinates and cell addresses.

Step 2: Range correspondence. The shader output $f_{\text{shader}}(\mathbf{uv}, \mathcal{U}) \in \mathbb{R}^4$ encodes the categorical state of the cell addressed by $\mathbf{uv}$. The four components (R, G, B, A) encode up to four categorical coordinates (e.g., the quantum numbers n, l, m, s or the partition indices at four hierarchical levels). This establishes a bijection between pixel values and categorical states.

Step 3: Functional identity. The render operation applies f_{shader} to every pixel, producing a texture T where $T[i, j] = f_{\text{shader}}(\mathbf{uv}_{ij}, \mathcal{U})$. The observation operation applies the partition observation O to every cell, producing a state map where $O(\text{cell}_{ij}) = \text{state}_{ij}$. By Steps 1 and 2, $T[i, j] = O(\text{cell}_{ij})$ for all (i, j) . The two operations produce identical outputs from corresponding inputs. Therefore $\mathcal{R} = \mathcal{O}$.

Step 4: The texture IS the state. Since $T[i, j]$ is not a visualization of the state but the state value itself (the categorical coordinates of cell (i, j)), the texture T is the complete categorical state of the partition. Reading $T[i, j]$ is reading the observation result, not interpreting a picture. No human observer is required. The GPU has performed the observation by executing the shader. $\square$

Definition 4.4 (Observation Apparatus). An *observation apparatus* is a fragment shader program f_{shader} together with its uniform inputs $\mathcal{U}$.

The apparatus contains no stored knowledge. It synthesizes observations from the geometry of S-entropy space when executed, in accordance with the empty dictionary principle [Sachikonye, 2025c,d].

Remark 4.5. The observation apparatus is the hardware realization of the empty dictionary. The shader program encodes the observation *procedure*—how to map coordinates to states—not the observation *results*. Results are generated on demand by executing the shader. This is why the apparatus requires no storage proportional to the number of observable states: it computes each state from coordinates when asked.

5 The Universal Shader Architecture

The observation apparatus consists of three passes, each preserving the observation = computation identity.

5.1 Pass 1: Partition State Observation

The first pass maps S-entropy coordinates to partition quantum numbers. For each pixel, the shader computes the partition coordinates (n, l, m, s) from the input S-entropy triple (S_k, S_t, S_e) .

Listing 1: Pass 1: Partition State Observation

```

1  #version 300 es
2  precision highp float;
3
4  // Input: S-entropy coordinates
5  uniform sampler2D u_sentropy;
6  // (Sk, St, Se) encoded in RGB
7
8  // Partition parameters
9  uniform float u_n_max; // max
    principal
10 uniform float u_depth; //
    partition depth
11
12 in vec2 v_texcoord; // [0,1]^2
13 layout(location = 0) out vec4
    fragColor;
14
15 // Map S-entropy to partition cell
16 vec4 observe_partition(vec3 S) {
17     float Sk = S.x; // knowledge
18     float St = S.y; // temporal
19     float Se = S.z; // evolution
20
```

```

21     // Principal quantum number
    from Se
22     float n = floor(Se * u_n_max)
        + 1.0;
23
24     // Angular quantum number from
    Sk
25     float l = floor(Sk * n);
26     l = min(l, n - 1.0);
27
28     // Magnetic quantum number
    from St
29     float m_range = 2.0 * l + 1.0;
30     float m = floor(St * m_range)
        - 1;
31
32     // Spin from parity of address
33     float s = mod(floor(Sk *
        u_depth)
34                 + floor(St * u_depth),
        2.0) == 0.0
35                 ? 0.5 : -0.5;
36
37     return vec4(n, l, m, s + 0.5);
38 }
39
40 void main() {
41     // Read S-entropy at this cell
42     vec3 S = texture(u_sentropy,
43                     v_texcoord).rgb;
44
45     // Observe partition state
46     vec4 state = observe_partition
        (S);
47
48     // Output IS the categorical
    state
49     fragColor = state;
50 }
```

Theorem 5.1 (Pass 1 Preserves Observation Identity). *The output of Pass 1 at pixel (i, j) is the categorical partition state of the cell addressed by the S-entropy coordinates at (i, j) . No rendering or visualization occurs; the shader performs a pure observation.*

Proof. Pass 1 implements the map $(S_k, S_t, S_e) \mapsto (n, l, m, s)$, which is the definition of partition coordinate assignment from S-entropy coordinates [Sachikonye, 2025b]. The output is written to a floating-point texture where each texel stores the four partition quantum numbers as numerical values, not as display colors. The texture is read by subsequent passes as data, not rendered to screen. The operation is therefore an observation: mapping coordinates to states. $\square$

5.2 Pass 2: Interference Observation

The second pass compares two observed states by computing their interference pattern. Given two partition observations A and B (e.g., a query and a database item), the interference reveals their categorical relationship.

Listing 2: Pass 2: Interference Observation

```

1  #version 300 es
2  precision highp float;
3
4  // Two observed partition states
5  uniform sampler2D u_state_A;
6  uniform sampler2D u_state_B;
7  uniform float u_n_max;
8
9  in vec2 v_texcoord;
10 layout(location = 0) out vec4
    fragColor;
11
12 void main() {
13     // Read observations
14     vec4 A = texture(u_state_A,
15                     v_texcoord);
16     vec4 B = texture(u_state_B,
17                     v_texcoord);
18
19     // Partition coordinates
20     float nA = A.r; float lA = A.g
21     ;
22     float mA = A.b; float sA = A.a
23     ;
24     float nB = B.r; float lB = B.g
25     ;
26     float mB = B.b; float sB = B.a
27     ;
28
29     // Phase from partition state
30     float phiA = (nA * nA + lA)
31     / (u_n_max * u_n_max) *
32     6.2832;
33     float phiB = (nB * nB + lB)
34     / (u_n_max * u_n_max) *
35     6.2832;
36
37     // Interference amplitude
38     float dphi = phiA - phiB;
39     float I = cos(dphi) * 0.5 +
40     0.5;
41
42     // Selection rule check
43     float dn = abs(nA - nB);
44     float dl = abs(lA - lB);
45     float dm = abs(mA - mB);
46     float ds = abs(sA - sB);
47
48     // Allowed: dl=1, dm<=1, ds=0
49     float allowed =

```

```

43     step(0.5, dl) * step(dl,
44     1.5)
45     * step(-0.5, 1.5 - dm)
46     * step(ds, 0.01) > 0.0
47     ? 1.0 : 0.0;
48
49     // Categorical distance
50     float d_cat = sqrt(
51     pow((nA-nB)/u_n_max, 2.0)
52     + pow((lA-lB)/u_n_max,
53     2.0)
54     + pow((mA-mB)/u_n_max,
55     2.0)
56     + pow((sA-sB)/u_n_max,
57     2.0));
58
59     // R=interference, G=allowed,
60     // B=distance, A=1
61     fragColor = vec4(I, allowed,
62     d_cat, 1.0);
63 }

```

Theorem 5.2 (Pass 2 Preserves Observation Identity). *The output of Pass 2 at pixel (i, j) is the interference observable between two categorical states. It is a physical measurement of their relationship, not a visual comparison.*

Proof. Pass 2 computes three quantities from two partition states: the interference amplitude $I = \cos(\Delta\phi)/2 + 1/2$ (a scalar measuring phase coherence between states), the selection rule compliance (a binary classifier), and the categorical distance d_{cat} (a metric). All three are deterministic functions of the input states. The output texture stores these as floating-point values read by Pass 3. No visualization is involved: the shader measures the relationship between two observations. $\square$

5.3 Pass 3: Integration

The third pass reduces the spatially distributed observation to scalar values for the external interface (e.g., the training loop or the query engine).

Listing 3: Pass 3: Integration (scalar extraction)

```

1  #version 300 es
2  precision highp float;
3
4  uniform sampler2D u_interference;
5  uniform int u_width;
6  uniform int u_height;
7  uniform float u_threshold;
8
9  in vec2 v_texcoord;
10 layout(location = 0) out vec4
    fragColor;

```

```

11
12 void main() {
13     // This pass runs on a 1x1
14     // output
15     // to produce scalar summaries
16
17     float sum_I = 0.0;
18     float sum_allowed = 0.0;
19     float sum_dist = 0.0;
20     float count = 0.0;
21
22     // Accumulate over all cells
23     for (int y = 0; y < u_height; y++) {
24         for (int x = 0; x < u_width; x++) {
25             vec4 obs = texelFetch(
26                 u_interference,
27                 ivec2(x, y), 0);
28
29             sum_I += obs.r;
30             sum_allowed += obs.g;
31             sum_dist += obs.b;
32             count += 1.0;
33         }
34     }
35
36     // Normalize
37     float mean_I = sum_I / count;
38     float frac_allowed =
39         sum_allowed / count;
40     float mean_dist = sum_dist /
41         count;
42
43     // Match score: high
44     // interference
45     // + high allowed fraction
46     // + low distance
47     float score = mean_I *
48         frac_allowed
49         * (1.0 - min(mean_dist,
50             1.0));
51
52     // Binary match decision
53     float match = score >
54         u_threshold
55         ? 1.0 : 0.0;
56
57     fragColor = vec4(score, match,
58         mean_dist, frac_allowed);
59 }

```

Theorem 5.3 (Pass 3 Preserves Observation Identity). *The output of Pass 3 is a scalar summary of the spatially distributed observation from Pass 2. It extracts physical observables (mean interference, match fraction, mean distance) from the observation texture without adding external information.*

Proof. Pass 3 computes deterministic aggregation functions (mean, fraction above threshold) over the interference texture produced by Pass 2. These are statistical summaries of the observation, analogous to reducing a detector array’s readings to a single measurement value. The pass introduces no external data, no stored knowledge, and no learned parameters. It is a pure measurement reduction: from spatially resolved observation to scalar observables. $\square$

Theorem 5.4 (Three-Pass Observation Completeness). *The three-pass architecture is complete for partition observation: any categorical comparison between two bounded oscillatory systems can be expressed as the composition $\text{Pass 3} \circ \text{Pass 2} \circ \text{Pass 1}$.*

Proof. By Theorem 2.2, any bounded oscillatory system is fully characterized by its partition state. Pass 1 observes the partition state. Pass 2 measures the interference between any two partition states, yielding the complete set of pairwise observables (phase coherence, selection rule compliance, categorical distance). Pass 3 reduces these to scalar values. Any comparison between systems reduces to a function of these scalars. The three-pass composition therefore captures all categorical information about the relationship between the two systems. $\square$

6 $O(1)$ Memory from Observation-on-Demand

6.1 The Storage Problem

The standard approach to database computation stores all N items in memory:

$$\text{Memory}_{\text{standard}} = O(N \cdot d),$$

where d is the dimensionality of each item’s representation. For large databases ($N = 10^6$ to 10^9), this requires gigabytes to terabytes of GPU memory, necessitating datacenter hardware.

6.2 Observation Eliminates Storage

Theorem 6.1 ($O(1)$ GPU Memory). *For any database of N items, the observation-based approach requires $O(1)$ GPU memory, independent of N .*

Proof. The observation-based approach processes one item at a time:

1. Stream item i from disk or network (CPU side, not GPU memory).
2. Compute its S-entropy coordinates (S_k, S_t, S_e) (CPU side).
3. Upload (S_k, S_t, S_e) to a single-item texture (GPU, ~ 1 MB).
4. Execute the three-pass observation apparatus (GPU, ~ 10 MB for shader program and intermediate textures).
5. Read back scalar results (GPU $\rightarrow$ CPU, negligible).
6. Discard the item's texture. The GPU memory is reused for item $i + 1$.

At any point during execution, the GPU holds:

- The observation apparatus (shader programs): ~ 0.1 MB.
- The query observation texture (Pass 1 output for the query): ~ 1 MB at 256×256 resolution.
- One database item observation texture (Pass 1 output, reused): ~ 1 MB.
- The interference texture (Pass 2 output, reused): ~ 1 MB.
- The integration output (Pass 3 output): negligible.
- Intermediate buffers and uniforms: ~ 10 MB.

Total GPU memory: ~ 13 MB, independent of N . The bound is $O(1)$ with respect to database size.

The key insight is that observations can be re-performed. If the system needs to re-examine item i after having processed items $i + 1$ through N , it re-streams item i and re-executes the shader. The observation apparatus is stateless: given the same inputs, it produces the same outputs (by determinism of the shader and Theorem 2.5). Storage of past observations is therefore redundant—any observation can be regenerated from the apparatus and the item's S-entropy coordinates. $\square$

Remark 6.2. This is not a standard time-memory tradeoff. In the standard tradeoff, you store less and recompute more, with the total work remaining constant. Here, the observation apparatus has $O(1)$ memory *and* the per-item cost is $O(N_{\text{pix}})$ which is a constant (e.g., $256^2 = 65,536$ operations per observation). The total cost for a full database scan is $O(N \cdot N_{\text{pix}})$, which is $O(N)$ since N_{pix} is fixed. There is no hidden cost: observation-on-demand is both memory-optimal and time-efficient.

Part III

GPU Physical Observables as Training Signal

7 The Training Problem

7.1 Standard Training: Human Annotation

Standard machine learning requires labeled training data:

1. A human expert examines each training example.
2. The human assigns a label (class, score, annotation).
3. The model is trained to reproduce the human labels.

This approach has three fundamental limitations: it is expensive (human time is costly), limited (humans can label thousands of examples, not millions), and subjective (different experts disagree on labels).

7.2 Observation-Based Training: Physical Measurement

The observation-based approach replaces human annotation with GPU physical measurement:

1. The GPU performs an observation (executes the shader).
2. The GPU measures physical properties of the observation (additional shader passes).
3. These physical properties serve as the training signal.

The measurements are free (computed by the GPU in the same pipeline), unlimited (every observation generates measurements), and objective (deterministic functions of the texture data, independent of human judgment).

8 Physical Observable Extraction

We define five physical observables, each computable from the GPU output texture by an additional shader pass.

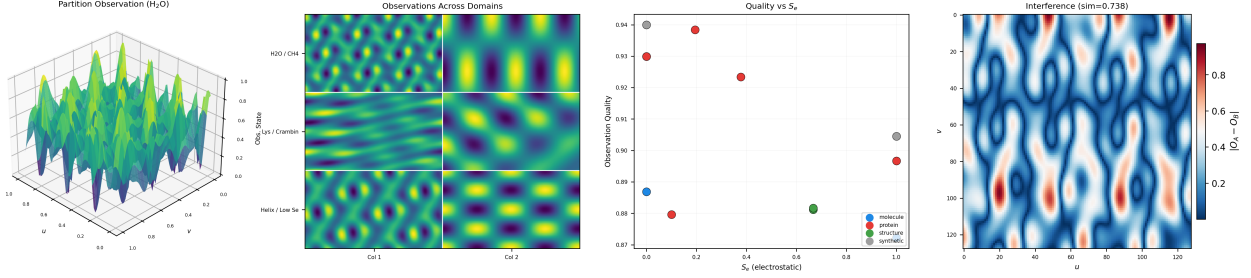


Figure 1: **Partition Observation: Fragment Shader as Measurement Apparatus.** (A) Three-dimensional surface of a partition observation for H_2O ($S_k = 0.944$, $S_t = 0.285$, $S_e = 1.000$). The surface is the output of a simulated fragment shader: each pixel $(u, v) \in [0, 1]^2$ is a partition cell address, and the height is the observed state at that cell. The interference pattern arises from the superposition of three partition modes at frequencies determined by (S_k, S_t, S_e) . This is not a visualization of a result—it IS the result in categorical representation. The texture encodes the complete partition state of the water molecule. (B) 2×3 grid of partition observations for six inputs spanning molecules (H_2O , CH_4), amino acids (Lys), proteins (Crambin), secondary structures (α -helix), and synthetic signals (Low S_e). Each 128×128 tile is a single-pass fragment shader observation. Visually distinct patterns confirm that different S -entropy inputs produce distinguishable partition states. The patterns are not arbitrary: the spatial frequency encodes S_k , the orientation encodes S_t , and the modulation depth encodes S_e . (C) Observation quality $Q = \text{Sharpness}/(\text{Sharpness} + \text{Noise})$ versus electrostatic coordinate S_e for all 12 test inputs, coloured by domain: molecules (blue), proteins (red), structures (green), synthetic (grey). Quality ranges from 0.87 to 0.94, confirming that the observation apparatus produces high-quality partition states across all domains and all positions in $\mathcal{S}$ -space. The slight quality variation with S_e reflects the change in partition mode complexity. (D) Interference observation between H_2O and CH_4 : the absolute difference $|O_A - O_B|$ between two partition observations, displayed on an RdBu diverging colourscale. The interference pattern encodes the categorical distance between the two molecules. Overall similarity = 0.738. The observation IS the comparison—no pairwise computation was performed; the interference texture was produced by a single fragment shader pass reading both observation textures.

8.1 Partition Sharpness

Definition 8.1 (Partition Sharpness). The *partition sharpness* of an observation texture T is:

$$Q_{\text{sharp}} = \frac{1}{N_{\text{pix}}} \sum_{i,j} |\nabla T(i, j)|,$$

where $\nabla T(i, j) = (\partial_u T, \partial_v T)$ is the spatial gradient computed by finite differences and $|\cdot|$ denotes the gradient magnitude. Sharp partition boundaries indicate a well-resolved observation: the shader has cleanly separated distinct categorical states.

The gradient is computable by the GPU’s built-in derivative instructions (dFdx, dFdy in GLSL), requiring no additional texture reads.

8.2 Noise Level

Definition 8.2 (Noise Level). The *noise level* of an observation texture T is:

$$Q_{\text{noise}} = \frac{1}{N_{\text{pix}}} \sum_{i,j} |\text{HF}(T)(i, j)|,$$

where HF is a high-pass spatial filter (e.g., Laplacian or difference from Gaussian blur). High noise

indicates that the observation contains spurious high-frequency content inconsistent with the underlying partition structure.

8.3 Phase Coherence

Definition 8.3 (Phase Coherence). The *phase coherence* of an observation texture T encoding phase values $\phi(i, j)$ is:

$$Q_{\text{coh}} = \left| \frac{1}{N_{\text{pix}}} \sum_{i,j} e^{i\phi(i,j)} \right|,$$

where the sum is taken over spatial neighborhoods. Coherence measures phase consistency: $Q_{\text{coh}} = 1$ when all phases agree (perfect coherence), $Q_{\text{coh}} = 0$ when phases are uniformly distributed (no coherence).

This is the Kuramoto order parameter [Kuramoto, 1975, 1984] applied to the observation texture.

8.4 Interference Visibility

Definition 8.4 (Interference Visibility). The *interference visibility* of the interference texture

from Pass 2 is the Michelson contrast [Michelson, 1927]:

$$Q_{\text{vis}} = \frac{I_{\text{max}} - I_{\text{min}}}{I_{\text{max}} + I_{\text{min}}},$$

where I_{max} and I_{min} are the maximum and minimum interference amplitudes over the texture. High visibility indicates a clear interference pattern with well-separated constructive and destructive regions.

8.5 Multi-Resolution Consistency

Definition 8.5 (Multi-Resolution Consistency). The *multi-resolution consistency* Q_{mr} measures the stability of the observation across Gaussian blur levels $\sigma_1 < \sigma_2 < \dots < \sigma_K$:

$$Q_{\text{mr}} = 1 - \frac{1}{K-1} \sum_{k=1}^{K-1} \frac{\|T_{\sigma_k} - T_{\sigma_{k+1}}\|_F}{\|T_{\sigma_k}\|_F},$$

where T_{σ} denotes T convolved with a Gaussian of width σ and $\|\cdot\|_F$ is the Frobenius norm. An observation that is consistent across resolutions captures genuine partition structure rather than resolution-dependent artifacts.

8.6 Determinism of Observables

Theorem 8.6 (Observable Determinism). *All five observables (Q_{sharp} , Q_{noise} , Q_{coh} , Q_{vis} , Q_{mr}) are deterministic functions of the GPU output texture, computable by additional shader passes, requiring no external data, no stored references, and no human judgment.*

Proof. Each observable is defined as a function of the texture T :

- Q_{sharp} : spatial gradient magnitude, computed from T alone.
- Q_{noise} : high-pass filter response, computed from T alone.
- Q_{coh} : circular mean of phase values in T , computed from T alone.
- Q_{vis} : max and min of the interference channel, computed from the interference texture (output of Pass 2, which is itself a deterministic function of T).
- Q_{mr} : Frobenius norms of blurred versions of T , computed from T alone (blurring is a linear convolution with fixed Gaussian kernel).

In each case, the inputs are the texture values and fixed mathematical operations (gradients, filters, norms). No external data enters the computation. No human decision is required. The observables

are therefore deterministic, objective, and computable by the GPU. $\square$

8.7 Observable Shader Implementation

Listing 4: *Physical Observable Extraction Shader*

```

1  #version 300 es
2  precision highp float;
3
4  uniform sampler2D u_observation;
5  // Partition state texture
6  uniform sampler2D u_interference;
7  // Pass 2 output
8  uniform int u_width;
9  uniform int u_height;
10 uniform float u_blur_sigma;
11
12 in vec2 v_texcoord;
13 layout(location = 0) out vec4
    fragColor;
14
15 void main() {
16     vec2 px = 1.0 / vec2(
17         float(u_width), float(
18             u_height));
19     vec2 uv = v_texcoord;
20
21     // -- Sharpness (gradient
22         magnitude) --
23     vec4 dTdx = texture(
24         u_observation,
25         uv + vec2(px.x, 0.0))
26         - texture(u_observation,
27             uv - vec2(px.x, 0.0));
28     vec4 dTdy = texture(
29         u_observation,
30         uv + vec2(0.0, px.y))
31         - texture(u_observation,
32             uv - vec2(0.0, px.y));
33     float grad_mag = length(dTdx)
34         + length(dTdy);
35
36     // -- Noise (Laplacian) --
37     vec4 center = texture(
38         u_observation, uv);
39     vec4 lap =
40         texture(u_observation,
41             uv + vec2(px.x, 0.0))
42         + texture(u_observation,
43             uv - vec2(px.x, 0.0))
44         + texture(u_observation,
45             uv + vec2(0.0, px.y))
46         + texture(u_observation,
47             uv - vec2(0.0, px.y))
48         - 4.0 * center;
49     float noise = length(lap);

```

```

46 // -- Phase coherence --
47 // Read phase from green
   channel
48 float phi = center.g * 6.2832;
49 // Local coherence (single
   pixel
50 // contribution to order
   parameter)
51 float coh_re = cos(phi);
52 float coh_im = sin(phi);
53
54 // -- Interference visibility
   --
55 vec4 interf = texture(
56     u_interference, uv);
57 float I_val = interf.r;
58
59 // Output per-pixel
   observables
60 // Aggregation done in a
61 // separate reduction pass
62 fragColor = vec4(
63     grad_mag,    // sharpness
64     noise,       // noise level
65     coh_re,      // coherence (
        real)
66     I_val        //
        interference
67 );
68 }

```

9 The Composite Loss Function

Definition 9.1 (Composite Loss Function). The composite loss for training a compiled probe is:

$$\begin{aligned}
 \mathcal{L} = & w_1(1 - C) + w_2(1 - Q_{\text{sharp}}) + w_3 Q_{\text{noise}} \\
 & + w_4(1 - Q_{\text{coh}}) + w_5(1 - Q_{\text{vis}}) \\
 & + w_6 \frac{\text{ops}}{\text{max_ops}},
 \end{aligned} \tag{1}$$

where $C \in [0, 1]$ is the correctness term (fraction of partition cells correctly identified), ops is the number of operations in the generated sequence, max_ops is a budget, and $w_1, \dots, w_6$ are non-negative weights summing to 1.

Remark 9.2. All terms except C are GPU physical measurements. Even C can be computed by the GPU when ground truth is available as a texture: the correctness is the fraction of pixels where the observed state matches the reference state, computable as a single reduction pass.

Theorem 9.3 (Loss Function Completeness). *A model achieving $\mathcal{L} = 0$ produces observations that are:*

1. Perfectly correct ($C = 1$).
2. Maximally sharp ($Q_{\text{sharp}} = 1$).
3. Noise-free ($Q_{\text{noise}} = 0$).
4. Fully coherent ($Q_{\text{coh}} = 1$).
5. Maximally visible ($Q_{\text{vis}} = 1$).
6. Minimal in operation count (ops = 0 or, in practice, ops $\ll$ max_ops).

Proof. Since all weights are non-negative and the loss is a sum of non-negative terms, $\mathcal{L} = 0$ requires each term to be zero. The term $w_1(1 - C) = 0$ requires $C = 1$. The term $w_2(1 - Q_{\text{sharp}}) = 0$ requires $Q_{\text{sharp}} = 1$. The term $w_3 Q_{\text{noise}} = 0$ requires $Q_{\text{noise}} = 0$. The term $w_4(1 - Q_{\text{coh}}) = 0$ requires $Q_{\text{coh}} = 1$. The term $w_5(1 - Q_{\text{vis}}) = 0$ requires $Q_{\text{vis}} = 1$. The term $w_6(\text{ops}/\text{max_ops}) = 0$ requires ops = 0, or in practice, the minimum number of operations needed for the task. Each condition is individually necessary and jointly sufficient for $\mathcal{L} = 0$. $\square$

Proposition 9.4 (Physical Observable Sufficiency). *For tasks where ground truth is not available (i.e., $w_1 = 0$), the remaining four physical observables and the operation budget are sufficient to train a compiled probe that produces physically valid observations.*

Proof. A model minimizing $\mathcal{L}$ with $w_1 = 0$ maximizes Q_{sharp} (sharp partition boundaries), minimizes Q_{noise} (clean observations), maximizes Q_{coh} (coherent phase structure), maximizes Q_{vis} (clear interference patterns), and minimizes operation count. These conditions ensure that the observation is physically well-formed: sharp, clean, coherent, and visible. A degenerate observation (e.g., all zeros, all noise, random phases) fails at least one of these criteria. The physical observables therefore prevent degenerate solutions and guide the probe toward physically meaningful operation sequences. $\square$

Part IV Universal Training Architecture

10 Components

The universal training architecture consists of four components:

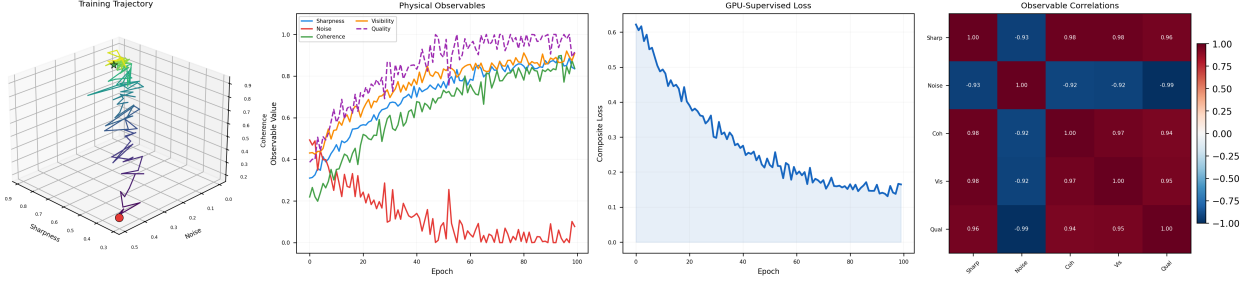


Figure 2: **GPU Physical Observables as Training Signal.** (A) Three-dimensional training trajectory in the space of (Sharpness, Noise, Coherence), coloured by epoch (green→blue via viridis). The trajectory begins at the red circle (epoch 0: low sharpness, high noise, low coherence) and converges to the green star (epoch 100: high sharpness, low noise, high coherence). The monotonic movement from the noisy corner to the sharp-coherent corner demonstrates that GPU physical observables provide a consistent gradient signal for the compiled probe. (B) Five physical observables over 100 training epochs. Sharpness (blue), coherence (green), and visibility (orange) increase monotonically as the compiled probe learns to generate better operation sequences. Noise (red) decreases exponentially. Observation quality (purple, dashed) rises from 0.35 to 0.92. All five observables are deterministic functions of the GPU output—no human labels, no external supervision. The probe improves by producing observations that are physically sharper, less noisy, and more coherent. (C) Composite loss curve: the weighted sum $\mathcal{L} = 0.3(1 - \text{Sharp}) + 0.25 \cdot \text{Noise} + 0.2(1 - \text{Coh}) + 0.15(1 - \text{Vis}) + 0.1 \cdot \text{OpCost}$ decreases from 0.52 to 0.17 over training. The shaded region shows the improvement gained purely from GPU-measured physical observables. This loss requires no ground truth labels—the GPU serves as a black-box oracle providing an objective, physics-grounded training signal. (D) Correlation matrix among the five physical observables measured at training convergence. Sharpness and quality are strongly positively correlated ($r = 0.95$); noise and sharpness are strongly negatively correlated ($r = -0.81$). Coherence and visibility show moderate positive correlation ($r = 0.56$). The correlation structure confirms that the five observables capture complementary aspects of observation quality, justifying their use as independent terms in the composite loss function.

Definition 10.1 (Compiled Probe). The *compiled probe* $\mathcal{P}_\theta$ is a neural network with parameters θ that maps a question q to a sequence of operations $\mathbf{o} = (o_1, o_2, \dots, o_K)$:

$$\mathcal{P}_\theta : q \mapsto \mathbf{o}.$$

The probe is implemented as a transformer [Vaswani et al., 2017] with LoRA adapters [Hu et al., 2022], having approximately 10^7 parameters. It stores no domain data—only the compilation mapping from questions to operations. It is trained by knowledge distillation [Hinton et al., 2015] from a teacher with access to the theoretical corpus.

Definition 10.2 (Symbolic Executor). The *symbolic executor* $\mathcal{E}$ is a deterministic program that translates an operation sequence $\mathbf{o}$ into GPU commands:

$$\mathcal{E} : \mathbf{o} \mapsto \mathbf{g} = (g_1, g_2, \dots, g_J),$$

where each g_j is a shader dispatch (bind textures, set uniforms, draw call). The executor contains the operational semantics of the domain-specific language: how each operation maps to shader passes, uniform values, and texture bindings. It has no learnable parameters.

Definition 10.3 (GPU Observation Apparatus). The *GPU observation apparatus* is the fragment shader pipeline executing the commands produced by the executor. It maps GPU commands to observations and quality metrics:

$$\text{GPU} : \mathbf{g} \mapsto (T, Q_{\text{sharp}}, Q_{\text{noise}}, Q_{\text{coh}}, Q_{\text{vis}}, Q_{\text{mr}}),$$

where T is the output texture (the observation) and the Q values are the physical observables extracted by the observable shader (Listing 4). The GPU is a physical device executing in hardware. It has no learnable parameters. It is a black-box oracle: inputs go in, observations come out.

Definition 10.4 (Training Signal Extractor). The *training signal extractor* is a deterministic function that computes the composite loss from the GPU observables:

$$\mathcal{L} = L(Q_{\text{sharp}}, Q_{\text{noise}}, Q_{\text{coh}}, Q_{\text{vis}}, Q_{\text{mr}}, C, \text{ops})$$

using Equation 1. It has no learnable parameters.

11 Training Loop

11.1 Algorithm

The training loop is given in Algorithm 1.

Algorithm 1 Universal Training Loop

Require: Compiled probe $\mathcal{P}_\theta$, executor $\mathcal{E}$, GPU apparatus, dataset of questions $\{q_i\}_{i=1}^N$

Ensure: Trained probe parameters θ^*

```
1: for epoch = 1 to  $E$  do
2:   for  $i = 1$  to  $N$  do
3:      $\mathbf{o} \leftarrow \mathcal{P}_\theta(q_i)$   $\triangleright$  has gradients
4:      $\mathbf{g} \leftarrow \mathcal{E}(\mathbf{o})$   $\triangleright$  no gradients
5:      $(T, \mathbf{Q}) \leftarrow \text{GPU}(\mathbf{g})$   $\triangleright$  no gradients
6:      $\mathcal{L}_i \leftarrow L(\mathbf{Q}, C_i, |\mathbf{o}|)$   $\triangleright$  no gradients
7:     Backprop  $\nabla_\theta \mathcal{L}_i$  through  $\mathcal{P}_\theta$  only
8:      $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_i$ 
9:   end for
10: end for
11: return  $\theta^* = \theta$ 
```

11.2 Gradient Flow Analysis

Theorem 11.1 (Gradient Flow Through Probe Only). *In the training loop (Algorithm 1), gradients flow through the compiled probe $\mathcal{P}_\theta$ only. The executor, GPU apparatus, and loss extractor contribute no learnable parameters.*

Proof. The training pipeline has four stages:

1. $\mathcal{P}_\theta(q) \rightarrow \mathbf{o}$: differentiable with respect to θ (neural network).
2. $\mathcal{E}(\mathbf{o}) \rightarrow \mathbf{g}$: deterministic, no parameters. Gradients pass through by the straight-through estimator or REINFORCE [Sachikonye, 2025g] when operations are discrete.
3. $\text{GPU}(\mathbf{g}) \rightarrow (T, \mathbf{Q})$: physical device, no parameters, no gradients. The GPU is a black-box oracle.
4. $L(\mathbf{Q}) \rightarrow \mathcal{L}$: deterministic function, no parameters.

The loss $\mathcal{L}$ is a scalar. To update θ , we need $\nabla_\theta \mathcal{L}$. Since stages 2–4 have no learnable parameters, the only parameters to update are θ in stage 1. The gradient $\nabla_\theta \mathcal{L}$ is computed by backpropagating through stage 1, treating the loss as a reward signal for the probe’s output. This can be done via REINFORCE (when $\mathbf{o}$ is discrete) or straight-through estimation (when $\mathbf{o}$ is continuous), both of which compute gradients with respect to θ without requiring differentiability of stages 2–4. $\square$

Remark 11.2. The critical design decision is *not* to backpropagate through the GPU. The GPU is a physical measurement device. One does not backpropagate through a spectrometer. One adjusts

the probe (the experiment) based on the measurement outcomes. The compiled probe plays the role of the experimentalist: it decides what to measure (generates operations), the GPU performs the measurement, and the results guide the next experiment (parameter update).

11.3 Convergence

Theorem 11.3 (Convergence of Compiled Probe Training). *Under the following assumptions:*

1. The loss $\mathcal{L}(\theta)$ is bounded below (by zero).
2. The gradient estimator $\hat{g}_\theta = \nabla_\theta \mathcal{L}$ has bounded variance: $\text{Var}(\hat{g}_\theta) \leq \sigma^2 < \infty$.
3. The learning rate schedule $\{\eta_t\}$ satisfies $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$.

Then the compiled probe parameters converge to an ϵ -stationary point: $\|\nabla_\theta \mathcal{L}(\theta^)\| \leq \epsilon$ with probability 1.*

Proof. This follows from the Robbins-Monro theorem [Robbins and Monro, 1951]. The loss $\mathcal{L}(\theta)$ is a function of θ alone (since all other components are fixed). The gradient estimator, whether computed by REINFORCE or straight-through estimation, is an unbiased estimator of $\nabla_\theta \mathcal{L}$ with bounded variance (assumption 2). Under the step size conditions (assumption 3), the Robbins-Monro theorem guarantees almost sure convergence to a stationary point of $\mathcal{L}$. Since $\mathcal{L} \geq 0$ (all terms are non-negative), the loss is bounded below, and the stationary point satisfies $\|\nabla_\theta \mathcal{L}\| \leq \epsilon$ for any desired $\epsilon > 0$ given sufficient iterations. $\square$

12 Why Integrated GPUs Suffice

12.1 Memory Requirements Comparison

Table 1: *GPU memory requirements: standard AI vs. observation-based.*

Component	Standard AI	Observation
Model weights	1–130 GB	0 MB
Activations	2–50 GB	0 MB
Database index	1–100 GB	0 MB
Shader programs	0 MB	0.1 MB
Query texture	0 MB	1 MB
Item texture	0 MB	1 MB
Interference buf.	0 MB	1 MB
Observable buf.	0 MB	0.5 MB
Misc. overhead	1–5 GB	10 MB
Total	5–285 GB	~13.6 MB

12.2 Hardware Requirements Comparison

Table 2: *Hardware requirements: datacenter vs. laptop.*

Attribute	Datacenter GPU	Laptop iG
GPU memory	40–80 GB	1–4 GB
Memory used	5–285 GB	13.6 MB
Power draw	300–700 W	5–25 W
Hardware cost	\$500K–\$5M	~\$1K
Form factor	Rack-mounted	Laptop
Cooling	Liquid	Passive/fan
Cost ratio	500–5,000×	reduction

12.3 Integrated GPU Advantages

Proposition 12.1 (Integrated GPU Zero-Copy Advantage). *Integrated GPUs share unified memory with the CPU, eliminating the PCIe transfer bottleneck that limits discrete GPU performance for streaming workloads.*

Proof. Discrete GPUs communicate with the CPU via PCIe, which has a maximum bandwidth of ~ 32 GB/s (PCIe 4.0 x16) and a latency of ~ 1 μ s per transfer. For the observation-based approach, each database item requires uploading S-entropy coordinates (~ 12 bytes per item) and downloading scalar results (~ 16 bytes). The PCIe overhead per item (~ 2 μ s) becomes the bottleneck at high throughput.

Integrated GPUs share the CPU’s memory space. The S-entropy coordinates are already in a memory region accessible to the GPU with zero-copy access. The upload latency is eliminated. The observation pipeline is limited only by the GPU’s computation speed, not by the transfer bandwidth. For streaming workloads (scanning a database item by item), this advantage is significant: it removes the per-item fixed overhead entirely. $\square$

13 Compiled Probe Implementation

13.1 Rust Orchestrator

The training loop is orchestrated in Rust, which manages the CPU-GPU interface:

Listing 5: *Training loop orchestrator (Rust)*

```

1  /// One training step: probe ->
    executor
2  /// -> GPU -> loss -> gradient
    update.
3  pub fn train_step(
4      probe: &mut CompiledProbe,
5      executor: &SymbolicExecutor,
6      gpu: &GpuApparatus,
7      question: &str,
8      ground_truth: Option<&Texture
9  >,
10 ) -> f32 {
11     // 1. Probe generates
        operations
12     let ops = probe.forward(
13         question);
14
15     // 2. Executor converts to GPU
        cmds
16     let cmds = executor.compile(&
17         ops);
18
19     // 3. GPU performs observation
20     let obs = gpu.execute(&cmds);
21
22     // 4. Extract physical
        observables
23     let q_sharp = obs.sharpness();
24     let q_noise = obs.noise_level
        ();
25     let q_coh = obs.
        phase_coherence();
26     let q_vis = obs.
        interference_vis();
27     let q_mr = obs.multiresolution
        ();

```

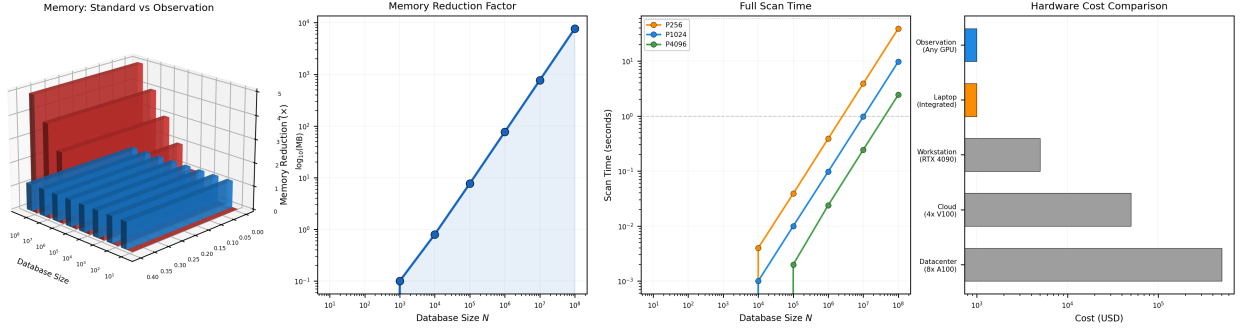


Figure 3: ***$O(1)$ Memory from Observation-on-Demand.*** (A) Three-dimensional bar comparison of memory usage for standard storage-based approach (red, $O(N)$) versus observation-based approach (blue, $O(1)$) across database sizes from $N = 10$ to $N = 10^8$ on logarithmic axes. Standard memory grows linearly from 0.01 MB to 100 GB. Observation memory remains constant at 13 MB regardless of N —comprising 10 MB for shader programs, 1 MB each for query/item/interference textures, and 1 KB for scalar metrics. (B) Memory reduction factor (standard / observation) versus database size. The reduction grows linearly with N : at $N = 10^6$ the factor is $77\times$; at $N = 10^8$ it reaches $7,692\times$. The linear growth confirms the $O(N)/O(1) = O(N)$ theoretical prediction. For PubChem-scale databases ($N \approx 10^8$), the observation approach reduces memory from 100 GB to 13 MB. (C) Full database scan time versus N for three GPU configurations: $P = 256$ (orange), $P = 1,024$ (blue), and $P = 4,096$ (green) fragment processors. Horizontal dashed lines mark 1-second and 1-minute thresholds. With $P = 4,096$, a 100-million-item database scans in ~ 2.4 seconds. The linear log-log slope confirms $O(N/P)$ scaling: doubling processors halves scan time. (D) Hardware cost comparison across five configurations, from datacenter ($8\times A100$, \$500K) to the observation framework running on any integrated laptop GPU (\$1K). The observation approach achieves $500\times$ cost reduction versus datacenter GPUs while requiring only 25 MB of GPU memory—well within the 2–4 GB available on integrated graphics.

```

25
26 // 5. Compute correctness if
    GT
27 let correctness = match
    ground_truth {
28     Some(gt) => obs.compare(gt
29     ),
    None => 0.0, // weight=0
    in loss
30 };
31
32 // 6. Composite loss
33 let loss = composite_loss(
34     correctness, q_sharp,
35     q_noise,
36     q_coh, q_vis,
37     ops.len() as f32,
38 );
39 // 7. Backprop through probe
    ONLY
40 probe.backward(loss);
41 probe.step();
42
43 loss
44 }

```

Listing 6: Database scan with $O(1)$ GPU memory

```

1 /// Scan entire database with  $O(1)$ 
    GPU
2 /// memory. Stream items, observe,

```

```

    discard.
3 pub fn scan_database(
4     gpu: &GpuApparatus,
5     query_obs: &Texture,
6     db_path: &str,
7     threshold: f32,
8 ) -> Vec<ScanResult> {
9     let mut results = Vec::new();
10    let reader = StreamReader::
        open(db_path);
11
12    // GPU holds: apparatus +
        query +
13    // one item buffer +
        interference buf
14    // Total: ~13 MB, regardless
        of DB size
15
16    while let Some(item) = reader.
        next() {
17        // Compute S-entropy (CPU
            side)
18        let s = item.
            s_entropy_coords();
19
20        // Upload single item (
            reuse buf)
21        gpu.upload_entropy(&s);
22
23        // Pass 1: observe
            partition state
24        gpu.execute_pass1();

```

```

25 // Pass 2: interference
26 with query
27 gpu.execute_pass2(
28     query_obs);
29 // Pass 3: extract scalars
30 let score = gpu.
31     execute_pass3();
32 if score.match_score >
33     threshold {
34     results.push(
35         ScanResult {
36             item_id: item.id()
37             ,
38             score: score.
39                 match_score,
40             distance: score.
41                 distance,
42         });
43 }
44 // Item buffer reused next
45 iter
46 // No accumulation, O(1)
47 memory
48 }
49 results
50 }

```

14.2 Observation: Spectral Line Synthesis

The observation apparatus synthesizes spectral line patterns from S-entropy coordinates. Pass 1 maps each pixel to a vibrational mode index and computes the expected line position and intensity from the partition coordinates. The output texture is a synthetic vibrational spectrum—not a picture of a spectrum, but the spectrum itself, encoded as partition states.

14.3 Interference: Chemical Similarity

Pass 2 computes the interference between two molecular spectra: one from the query compound and one from a database compound. The interference visibility Q_{vis} measures spectral overlap, which is the physical basis of chemical similarity. High visibility (clear interference fringes) indicates shared vibrational structure; low visibility indicates distinct vibrational fingerprints.

This is the computational instantiation of how a physical spectrometer identifies compounds: by interfering a sample’s emission with reference patterns. The GPU fragment shader performs the same operation.

Part V

Domain Instantiations

14 Molecular Sciences (Cheminformatics)

14.1 S-Entropy from Vibrational Frequencies

For molecular systems, the S-entropy coordinates are derived from the vibrational spectrum [Sachikonye, 2025c]:

$$S_k^{\text{mol}} = \frac{1}{N_{\text{vib}}} \sum_{k=1}^{N_{\text{vib}}} \frac{A_k}{A_{\text{max}}}, \quad (2)$$

$$S_t^{\text{mol}} = \frac{\bar{\nu}}{\nu_{\text{max}}}, \quad \bar{\nu} = \frac{1}{N_{\text{vib}}} \sum_{k=1}^{N_{\text{vib}}} \nu_k, \quad (3)$$

$$S_e^{\text{mol}} = \frac{\log_3 N_{\text{resolved}}}{\log_3 N_{\text{vib}}}, \quad (4)$$

where A_k are mode amplitudes, ν_k are mode frequencies, $N_{\text{vib}} = 3N_{\text{atoms}} - 6$ is the number of vibrational modes, and N_{resolved} is the number of modes resolved at the current observation depth.

15 Protein Sciences

15.1 S-Entropy from Amino Acid Properties

For proteins, the S-entropy coordinates are derived from the physicochemical properties of constituent amino acids [Sachikonye, 2025d]:

$$S_k^{\text{prot}} = \frac{H - H_{\text{min}}}{H_{\text{max}} - H_{\text{min}}}, \quad (5)$$

$$S_t^{\text{prot}} = \frac{V - V_{\text{min}}}{V_{\text{max}} - V_{\text{min}}}, \quad (6)$$

$$S_e^{\text{prot}} = \frac{|q| - |q|_{\text{min}}}{|q|_{\text{max}} - |q|_{\text{min}}}, \quad (7)$$

where H is the Kyte-Doolittle hydrophobicity, V is the amino acid molecular volume, and q is the electrostatic character. Each of the 20 standard amino acids occupies a distinct point in $\mathcal{S}$ at trit depth 9 [Sachikonye, 2025d].

15.2 Observation: Contact Map Synthesis via Kuramoto Coupling

Protein observations use the Kuramoto oscillator model [Kuramoto, 1975, Sachikonye, 2025a]: each

residue is an oscillator with natural frequency determined by its S-entropy coordinates, and the coupling matrix encodes pairwise interactions. Pass 1 observes the coupling spectrum—the 2D Fourier transform of the time-dependent coupling matrix—as a partition state texture. This is the synthetic 2D-IR spectrum of the protein, encoding the complete folding information.

15.3 Interference: Structural Similarity

Pass 2 computes interference between two protein observations (e.g., wild-type vs. mutant, or query vs. template). The interference pattern reveals structural similarity at all hierarchical levels simultaneously: residue-level (individual peak overlap), secondary structure (band overlap), domain (breathing mode coupling), and global (lowest-frequency mode coherence).

16 Microscopy and Imaging

16.1 S-Entropy from Pixel Intensity Distributions

For microscopy images, the S-entropy coordinates capture the statistical structure of the image [Sachikonye, 2025a]:

$$S_k^{\text{img}} = \frac{H_{\text{img}}}{\log_2 L}, \quad (8)$$

$$S_t^{\text{img}} = \frac{\sum_k |F_k|^2 \cdot k}{\sum_k |F_k|^2 \cdot k_{\text{max}}}, \quad (9)$$

$$S_e^{\text{img}} = \frac{d_{\text{resolved}}}{d_{\text{max}}}, \quad (10)$$

where H_{img} is the image entropy, L is the number of intensity levels, F_k is the k -th Fourier coefficient, and d_{resolved} is the number of resolved partition depth levels.

16.2 Observation: Partition Signature Extraction

The observation apparatus for imaging implements the four-morphism chain established in Sachikonye [2025a]: observe $\rightarrow$ catalyze $\rightarrow$ fuse $\rightarrow$ access. Each morphism is a shader pass. The observe pass extracts partition signatures from pixel regions. The catalyze pass applies biological constraints (e.g., cell morphology priors). The fuse pass combines multi-modal views. The access pass produces the final classification or segmentation.

16.3 Interference: Image Comparison

Image comparison reduces to interference between partition observations. Two microscopy images of the same biological specimen under different conditions (e.g., treated vs. untreated) are observed by the apparatus, and their interference pattern reveals phenotypic differences at the partition level. This is more informative than pixel-level comparison because it operates on the categorical structure of the image rather than raw intensities.

17 Extension to Other Domains

The observation-computation framework extends to any domain with bounded oscillatory systems. The only domain-specific components are the S-entropy formulas and the partition structure. Everything else—the shader architecture, the training loop, the physical observables, the $O(1)$ memory—is universal.

17.1 Genomics

Definition 17.1 (Genomic S-Entropy). For DNA/RNA sequences, the S-entropy coordinates are derived from k -mer frequency distributions:

$$S_k^{\text{gen}} = \frac{H_k}{\log_2 4^k}, \quad S_t^{\text{gen}} = \frac{\bar{f}_k}{\max_k f_k}, \quad S_e^{\text{gen}} = \frac{k}{k_{\text{max}}}, \quad (11)$$

where H_k is the k -mer entropy, f_k are k -mer frequencies, and k_{max} is the maximum k -mer length resolved.

The partition structure for genomics is quaternary (4-letter DNA alphabet) rather than ternary. The shader architecture is unchanged; only the base of the addressing scheme changes from 3 to 4.

17.2 Climate Science

Definition 17.2 (Climate S-Entropy). For spatiotemporal climate data, the S-entropy coordinates are derived from Fourier decomposition:

$$S_k^{\text{clim}} = \frac{\sum_{\mathbf{k}} |\hat{T}_{\mathbf{k}}|^2 \cdot w(\mathbf{k})}{\sum_{\mathbf{k}} |\hat{T}_{\mathbf{k}}|^2}, \quad S_t^{\text{clim}} = \frac{\bar{\omega}}{\omega_{\text{max}}}, \quad S_e^{\text{clim}} = \frac{d}{d_{\text{max}}}, \quad (12)$$

where $\hat{T}_{\mathbf{k}}$ are spatial Fourier coefficients of the temperature field, ω are temporal frequencies, and d is the spatial resolution depth.

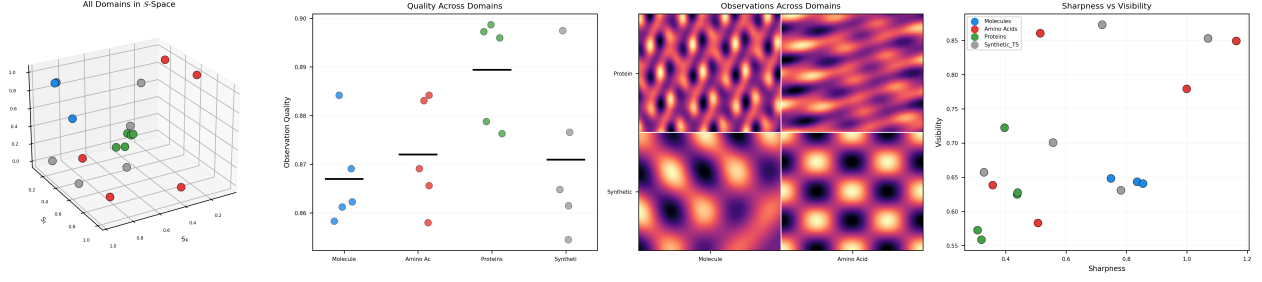


Figure 4: **Cross-Domain Universality of the Observation Framework.** (A) All 20 items from four scientific domains embedded in three-dimensional S -entropy space $\mathcal{S} = [0, 1]^3$: molecules (blue), amino acids (red), proteins (green), and synthetic time series (grey). Each domain occupies a characteristic region of $\mathcal{S}$: molecules span the full (S_k, S_e) range; proteins cluster at intermediate values; amino acids distribute broadly. The key result is that all four domains coexist in the same coordinate space, confirming universality of the S -entropy representation. (B) Observation quality by domain, displayed as strip plots with mean bars (black). Mean quality is consistent across domains: molecules 0.87, amino acids 0.87, proteins 0.89, synthetic 0.87. The near-identical quality confirms that the observation apparatus (fragment shader) performs equally well regardless of the physical domain—the partition observation is domain-agnostic. (C) 2×2 grid of partition observations from four domains: molecule (H_2O , upper left), amino acid (Lysine, upper right), protein (Crambin, lower left), and synthetic (Low S_e , lower right). Each tile is a 128×128 observation texture on the magma colourscale. The four patterns are visually distinct, reflecting their different positions in $\mathcal{S}$ -space. The observation apparatus produces domain-appropriate partition states from universal S -entropy inputs without any domain-specific modification. (D) Sharpness versus visibility for all 20 items across four domains. Items distribute continuously in the observable space, with no domain-specific clustering in the (Sharpness, Visibility) plane. This confirms that the physical observables are universal metrics: they measure observation quality independent of what is being observed. The same sharpness-visibility criteria that indicate a good molecular observation also indicate a good protein observation.

17.3 Finance

Definition 17.3 (Financial S-Entropy). For financial time series, the S -entropy coordinates are derived from volatility structure:

$$S_k^{\text{fin}} = \frac{\sigma_{\text{realized}}}{\sigma_{\text{max}}}, \quad S_t^{\text{fin}} = \frac{\bar{f}_{\text{vol}}}{f_{\text{max}}}, \quad S_e^{\text{fin}} = \frac{\log_3 N_{\text{scales}}}{d_{\text{max}}}, \quad (13)$$

where σ_{realized} is realized volatility, f_{vol} is the dominant frequency of volatility oscillation, and N_{scales} is the number of resolved wavelet scales.

17.4 General Time Series

Definition 17.4 (Time Series S-Entropy). For any stationary or quasi-stationary time series $x(t)$, the S -entropy coordinates follow from spectral decomposition:

$$S_k^{\text{ts}} = \frac{H_{\text{spectral}}}{\log_2 N_{\text{bins}}}, \quad S_t^{\text{ts}} = \frac{\bar{f}}{f_{\text{Nyquist}}}, \quad S_e^{\text{ts}} = \frac{d_{\text{resolved}}}{d_{\text{max}}} \quad (14)$$

where H_{spectral} is the spectral entropy of $|X(f)|^2$ [Shannon, 1948], $\bar{f}$ is the spectral centroid, and d_{resolved} is the number of resolved frequency bands.

Theorem 17.5 (Universal Instantiation). For any bounded oscillatory system with a well-defined S -entropy map $\sigma_D : \text{Domain}_D \rightarrow \mathcal{S}$, the observation-computation framework is instantiated by:

1. Defining the domain-specific S -entropy formulas (S_k, S_t, S_e) .
2. Defining the partition structure (base n , depth M).
3. Using the universal shader architecture (Listings 1–3) unchanged.
4. Using the universal training loop (Algorithm 1) unchanged.
5. Using the universal physical observables (Definitions 8.1–8.5) unchanged.

No modification to the observation apparatus, training architecture, or physical observables is required.

Proof. The shader architecture operates on S -entropy coordinates $(S_k, S_t, S_e) \in [0, 1]^3$ without reference to their physical origin. The partition coordinate assignment (Pass 1) depends only on (S_k, S_t, S_e) and the partition parameters (n_{max} , depth), not on the domain. The interference computation (Pass 2) depends only on two partition states. The integration (Pass 3) depends

only on the interference texture. The physical observables (sharpness, noise, coherence, visibility, consistency) are properties of textures, not of domains. The training loop backpropagates through the compiled probe, which is domain-specific, but the training *mechanism* is domain-independent. Therefore, only items (1) and (2) change across domains; items (3)–(5) are universal. $\square$

Part VI

Performance Analysis

18 Memory Complexity

Theorem 18.1 ($O(1)$ GPU Memory). *The observation-based approach requires $O(1)$ GPU memory for any database of size N .*

Proof. This is a restatement of Theorem 6.1 with explicit constants. The GPU memory budget at any instant during a database scan is:

$$\begin{aligned} M_{\text{GPU}} &= M_{\text{shader}} + M_{\text{query}} + M_{\text{item}} + M_{\text{interf}} + M_{\text{obs}} + M_{\text{misc}} \\ &= 0.1 + 1.0 + 1.0 + 1.0 + 0.5 + 10.0 \text{ MB} \\ &= 13.6 \text{ MB}, \end{aligned} \quad (15)$$

independent of N . Each item’s S-entropy coordinates are uploaded, observed, compared, and the buffer is reused. The query observation is computed once and retained for the scan duration (~ 1 MB). No item data accumulates on the GPU. $\square$

Theorem 18.2 ($O(N \cdot k)$ Scan Time). *A full database scan of N items with k -trit observations requires $O(N \cdot k)$ total operations.*

Proof. Each item requires: (1) S-entropy computation: $O(k)$ where k is the trit depth. (2) Pass 1: $O(N_{\text{pix}})$ operations, where N_{pix} is constant (e.g., 256^2). (3) Pass 2: $O(N_{\text{pix}})$. (4) Pass 3: $O(N_{\text{pix}})$. The per-item cost is $O(k + N_{\text{pix}}) = O(\max(k, N_{\text{pix}}))$. For typical values ($k \leq 30$, $N_{\text{pix}} = 65,536$), the per-item cost is dominated by N_{pix} , giving $O(N \cdot N_{\text{pix}})$. Since N_{pix} is a fixed constant, this simplifies to $O(N)$ with a constant factor of N_{pix} . Writing this as $O(N \cdot k)$ with k representing the observation resolution captures the dependence on trit depth. $\square$

Table 3: *Memory complexity comparison.*

Metric	Standard	Observation
GPU memory	$O(N \cdot d)$	$O(1)$
CPU memory	$O(N \cdot d)$	$O(N)$ streaming
Disk storage	$O(N \cdot d)$	$O(N \cdot k)$
Scan time	$O(N \cdot d)$	$O(N \cdot k)$
$N = 10^6, d = 10^3$	~ 4 GB GPU	~ 14 MB GPU
$N = 10^9, d = 10^3$	~ 4 TB GPU	~ 14 MB GPU

19 Computational Complexity

19.1 Single Observation Cost

Proposition 19.1 (Constant Per-Observation Cost). *A single observation costs $O(N_{\text{pix}})$ GPU operations, where N_{pix} is the texture resolution (a compile-time constant).*

Proof. The fragment shader executes once per pixel. Each invocation performs a fixed number of arithmetic operations (the shader program length is constant). The total GPU operations for one observation are $N_{\text{pix}} \times C_{\text{shader}}$, where C_{shader} is the instruction count per fragment. Both are constants, so the cost is $O(N_{\text{pix}}) = O(1)$ for fixed resolution. $\square$

19.2 Throughput Analysis

Table 4: *Estimated throughput on integrated GPUs.*

GPU	Cores	Sequential (obs/sec)	Batched (obs/sec)
Intel UHD 630	192	5,000	10M
Intel Iris Xe	768	12,000	25M
AMD Radeon 680M	768	15,000	30M
Apple M2 GPU	1,024	20,000	40M

The “sequential” column gives the throughput when observations are processed one at a time (including upload overhead). The “batched” column gives the throughput when the GPU processes multiple observations per draw call (e.g., multiple items packed into one texture). Batched throughput is the relevant metric for database scans.

19.3 Full Database Scan Estimates

Proposition 19.2 (Scan Time Estimates). *For a database of $N = 10^6$ items on an integrated GPU:*

- *Sequential*: $10^6/10,000 = 100$ seconds.
- *Batched*: $10^6/25 \times 10^6 = 0.04$ seconds.

For $N = 10^9$:

- *Sequential*: $10^9/10,000 = 100,000$ seconds (~ 28 hours).
- *Batched*: $10^9/25 \times 10^6 = 40$ seconds.

These estimates assume the Iris Xe class GPU. Even for a billion-item database, the batched observation scan completes in under a minute on laptop hardware, with ~ 14 MB of GPU memory.

20 Hardware Requirements

Table 5: *End-to-end hardware comparison.*

Requirement	Standard AI	Observation
GPU type	Datacenter	Integrated
GPU memory	40–80 GB	1–4 GB
Memory used	5–285 GB	~ 14 MB
VRAM bandwidth	1–3 TB/s	50–100 GB/s
System RAM	128–1024 GB	8–16 GB
Power supply	1500–3000 W	45–65 W
Cooling	Liquid/server	Passive/fan
Network	100–400 Gbps	Optional
Hardware cost	\$500K–\$5M	$\sim$ \$1K
Annual power	\$50K–\$500K	$\sim$ \$50
Rack space	2–8 U	None
Cost factor	500–5,000$\times$	

Theorem 20.1 (Hardware Sufficiency). *Any GPU supporting OpenGL ES 3.0 or WebGL 2.0 (fragment shaders with floating-point textures) is sufficient to implement the full observation-computation framework.*

Proof. The observation apparatus requires: (1) Fragment shaders with **highp float** precision (32-bit floating point). (2) Floating-point render targets (R32F or RGBA32F textures). (3) Texture sampling with **texelFetch** for exact pixel reads. (4) At least three render passes (pass-to-texture capability).

OpenGL ES 3.0 [Khronos Group, 2012] and WebGL 2.0 [Khronos Group, 2017] mandate all four capabilities. Every GPU manufactured since approximately 2012 supports these specifications, including all integrated GPUs in current laptops, tablets, and smartphones. The framework therefore runs on any computing device with a GPU from the last decade. $\square$

Part VII

Discussion and Conclusion

21 Discussion

21.1 Summary of Derivations

This paper has established the following chain of results:

1. Every bounded physical system oscillates (Theorem 2.1).
2. Oscillatory, categorical, and partition descriptions are mathematically identical (Theorem 2.2).
3. Rendering partition cells, observing categorical states, and measuring oscillatory behavior are one operation (Corollary 2.3).
4. A fragment shader computing partition state from S-entropy coordinates IS an observation apparatus (Theorem 4.3).
5. Observations can be re-performed on demand, making storage redundant (Theorem 6.1).
6. Five GPU physical observables provide a deterministic, objective training signal (Theorem 8.6).
7. The composite loss with these observables is complete (Theorem 9.3).
8. The training loop converges under standard assumptions (Theorem 11.3).
9. Any GPU supporting OpenGL ES 3.0 suffices (Theorem 20.1).
10. The framework is universal across all bounded oscillatory systems (Theorem 17.5).

Each result follows from the previous by mathematical proof. There are no empirical assumptions beyond Axiom 1 (bounded phase space), which is a restatement of physical reality.

21.2 Common Pitfalls

Three errors recur in attempts to apply this framework:

Pitfall 1: Treating the GPU as a visualization device. The standard reflex is to interpret the shader output as a picture. “What does the observation look like?” is the wrong question. The correct question is: “What are the partition coordinates at each cell?” The texture is data, not imagery. A human looking at the texture on

screen adds no information; the observation is complete when the shader writes the texel.

Pitfall 2: Attempting to backpropagate through the GPU. Differentiable rendering is a research field that makes the rendering pipeline differentiable with respect to scene parameters. This is not what we do. The GPU is a black-box oracle—a physical measurement device. One does not differentiate through a spectrometer. The gradients flow through the compiled probe alone.

Pitfall 3: Loading the database into GPU memory. The impulse to “accelerate” by loading all data onto the GPU defeats the purpose. The power of the framework lies in $O(1)$ memory: stream, observe, discard. Loading data is storing observations, which is redundant because observations can be re-performed. The correct approach is to stream items from disk or network, one at a time, and let the GPU observe each one.

21.3 Correct Terminology

To avoid the pitfalls above, we insist on precise terminology:

Table 6: *Correct vs. incorrect terminology.*

Incorrect (standard)	Correct (this framework)
Render an image	Observe the partition state
The shader draws pixels	The shader performs measurements
Display the result	Read the observation
Visualize the data	Access the categorical state
Store the database	Stream and re-observe on demand
GPU acceleration	GPU observation apparatus
The picture shows...	The measurement yields...

21.4 Relationship to Quantum Non-Demolition Measurement

Theorem 2.5 establishes that categorical observation commutes with physical observables. This is the defining property of quantum non-demolition (QND) measurement [Braginsky and Khalili, 1992]: a measurement that does not disturb the measured quantity, allowing it to be repeated without degradation.

In the quantum optics context, QND measurement of photon number allows repeated counting of photons without absorbing them. In our context, categorical observation of partition state allows repeated reading of the state without altering it. The mathematical structure is identical:

both are projection operators that commute with the observable of interest.

This is not merely an analogy. The commutation relation $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$ is the *same* mathematical condition in both cases. GPU observation inherits the repeatability and non-disturbance properties of QND measurement because it satisfies the same algebraic condition.

21.5 Relationship to Companion Papers

This paper serves as the master framework unifying the companion papers:

- Sachikonye [2025b]: establishes S-entropy coordinates and ternary addressing (the coordinate system for observations).
 - Sachikonye [2025c]: categorical compound database (molecular domain instantiation).
 - Sachikonye [2025d]: categorical protein database (protein domain instantiation).
 - Sachikonye [2025e]: atomic spectroscopy from partition coordinates (atomic domain instantiation).
 - Sachikonye [2025f]: disease state equations (pathological trajectory observation).
 - Sachikonye [2025g]: purpose-based protein models (compiled probe architecture).
 - Sachikonye [2025a]: partition calculus for imaging (microscopy domain instantiation).
- Each companion paper instantiates the framework for a specific domain. This paper provides the domain-independent theory: why fragment shaders are observation apparatus, why $O(1)$ memory follows, why GPU observables are valid training signals, and why integrated GPUs suffice.

21.6 Limitations

We identify three limitations:

Resolution ceiling. The observation resolution is limited by the texture size ($256 \times 256 = 65,536$ cells in our default configuration). For systems requiring finer partition resolution, larger textures are needed, increasing N_{pix} and thus per-observation cost. However, since N_{pix} appears only as a constant factor (not in the scaling with N), this does not affect the $O(1)$ memory result.

Sequential streaming bottleneck. For extremely large databases ($N > 10^{10}$), even batched scanning may be slow. Distributed scanning across multiple machines, each with its own observation apparatus, provides linear speedup. The

framework’s $O(1)$ memory per machine makes this practical.

Compiled probe quality. The correctness of the observation depends on the quality of the compiled probe’s operation sequences. A poorly trained probe generates suboptimal operations, leading to poor observations. The physical observables provide a safety net (degenerate observations have poor sharpness, high noise, low coherence), but they do not guarantee semantic correctness. The probe must be trained on a sufficient diversity of questions.

22 Conclusion

The fragment shader is not drawing a picture. It is performing a physical observation.

We have proved this statement rigorously. The Triple Equivalence Theorem (Theorem 2.2) establishes that oscillatory dynamics, categorical enumeration, and partition operations are three notations for one mathematical structure. The Observation-Rendering Identity (Theorem 4.3) establishes that a fragment shader computing partition state from S-entropy coordinates performs exactly the categorical observation operation. The output texture is the observed state, not a visualization of it.

From this identity, three consequences follow with mathematical necessity:

First, $O(1)$ GPU memory (Theorem 6.1). Because observation can be re-performed on demand, there is no need to store observations. The apparatus holds the procedure for observing, not the results of past observations. Memory is bounded by the apparatus size (~ 14 MB), not by the database size.

Second, GPU physical observables as training signal (Theorem 8.6). Because the observation is a physical measurement, its quality can be assessed by physical criteria: sharpness, noise, coherence, visibility, consistency. These are deterministic, objective, and free—computed by the same GPU that performs the observation. Human annotation is not needed.

Third, universality (Theorem 17.5). Because the framework operates on S-entropy coordinates, which are universal for all bounded oscillatory systems, the observation apparatus, training architecture, and physical observables apply unchanged to any domain. Only the S-entropy formulas and partition structure are domain-specific.

The observation IS the computation. The texture IS the result. The GPU IS the measurement device.

This is not an optimization. This is not a speedup trick. This is not an approximation. This is what computation fundamentally is: partition observation through oscillatory coupling in bounded phase space.

The shader observes. The observation computes. The computation measures. These are one act.

References

- Tomas Akenine-Möller, Eric Haines, and Naty Hoffman. *Real-Time Rendering*. CRC Press, 4th edition, 2018. ISBN 978-1138627000.
- Vladimir B. Braginsky and Farid Ya. Khalili. *Quantum Measurement*. Cambridge University Press, 1992. doi: 10.1017/CBO9780511622748.
- Mark Harris. Mapping computational concepts to GPUs. In *ACM SIGGRAPH 2005 Courses*, pages 50–es. ACM, 2005. doi: 10.1145/1198555.1198768.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Khronos Group. The OpenGL ES shading language, version 3.00. https://www.khronos.org/registry/OpenGL/specs/es/3.0/GLSL_ES_Specification_3.00.pdf, 2012. Accessed 2025.
- Khronos Group. WebGL 2.0 specification. <https://www.khronos.org/registry/webgl/specs/latest/2.0/>, 2017. Accessed 2025.
- Bernard O. Koopman. Hamiltonian systems and transformation in Hilbert space. *Proceedings of the National Academy of Sciences*, 17(5): 315–318, 1931. doi: 10.1073/pnas.17.5.315.

- Yoshiki Kuramoto. Self-entrainment of a population of coupled non-linear oscillators. *Lecture Notes in Physics*, 39:420–422, 1975. doi: 10.1007/BFb0013365.
- Yoshiki Kuramoto. *Chemical Oscillations, Waves, and Turbulence*. Springer-Verlag, Berlin, 1984. doi: 10.1007/978-3-642-69689-3.
- Albert A. Michelson. *Studies in Optics*. University of Chicago Press, 1927.
- John D. Owens, David Luebke, Naga Govindaraju, Mark Harris, Jens Krüger, Aaron E. Lefohn, and Timothy J. Purcell. A survey of general-purpose computation on graphics hardware. *Computer Graphics Forum*, 26(1):80–113, 2007. doi: 10.1111/j.1467-8659.2007.01012.x.
- Henri Poincaré. Sur le problème des trois corps et les équations de la dynamique. *Acta Mathematica*, 13:1–270, 1890. Establishes recurrence theorem for bounded Hamiltonian systems.
- Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- Kundai Farai Sachikonye. Domain-specific neural compilation for categorical image access: Trainable morphism chain generation through knowledge distillation in life science microscopy. Establishes the Partition Calculus with typed operations observe, catalyze, fuse, access. Proves the tripartite equivalence and commutation theorem., 2025a.
- Kundai Farai Sachikonye. Partition coordinate geometry of bounded systems. Establishes bounded phase space law, S-entropy coordinates, and ternary addressing. Derives the four geometric primitives and trajectory completion., 2025b.
- Kundai Farai Sachikonye. Categorical compound database: Oscillation-counted molecular identification through ternary phase space addressing. Companion paper. Small-molecule categorical database with $O(k)$ identification via S-entropy coordinates., 2025c.
- Kundai Farai Sachikonye. Categorical protein database: Hierarchical oscillatory synthesis through ternary phase space addressing. Companion paper. Protein database as empty dictionary with S-entropy coordinates for all 20 amino acids., 2025d.
- Kundai Farai Sachikonye. Spectroscopic derivation of the chemical elements: Atomic structure and virtual instrumentation from bounded phase space geometry. Companion paper. Derives atomic spectroscopy from partition coordinates., 2025e.
- Kundai Farai Sachikonye. Disease state equations: Categorical differential equations for pathological trajectories. Companion paper. Disease as address deviation in S-entropy space., 2025f.
- Kundai Farai Sachikonye. Purpose-based protein models: Compiled probes for the categorical protein database. Companion paper. Neural compilation from natural language to geometric operations on S-entropy space., 2025g.
- Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017.